LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASICS WITH COMPOSE

Oleh:

Arinda Dita Pratiwi NIM. 2410817120018

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basics with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Arinda Dita Pratiwi
NIM : 2410817120018

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 9
B. Output Program 14
C. Pembahasan 17
D. Essay 19
E. Daftar Pustaka 22
TAUTAN GIT 24

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 6
Gambar 2. Tampilan Dadu Setelah Di-Roll..... 7
Gambar 3. Tampilan Roll Dadu Double 7
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML..... 14
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML..... 14
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML..... 15
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose..... 15
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose..... 16
Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose..... 16

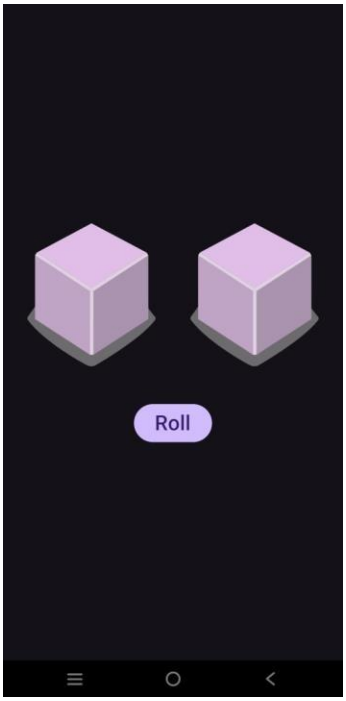
DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt XML 9
Tabel 2. Source Code activity_main.xml..... 10
Tabel 3. Source Code MainActivity.kt Compose 11

SOAL 1

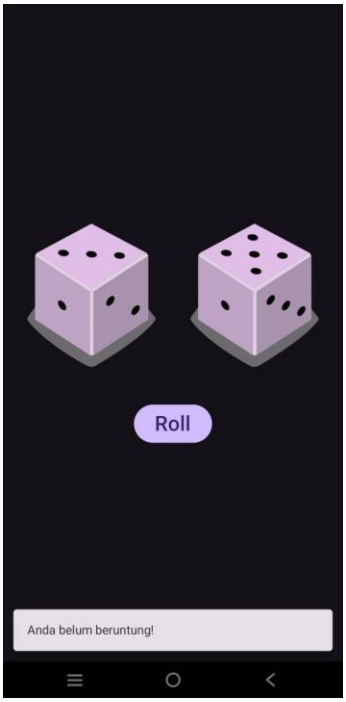
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



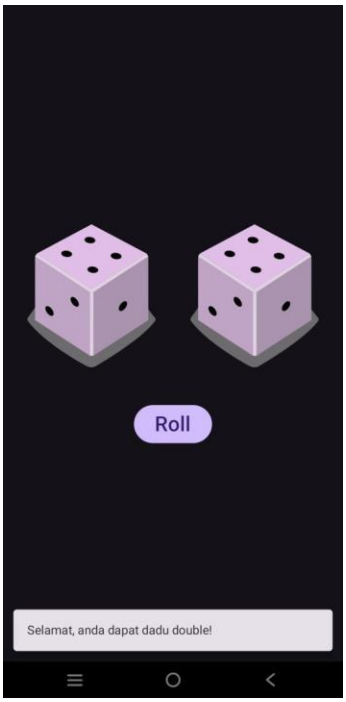
Gambar 1. Tampilan Awal Aplikasi

2. Tampilan setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan "Selamat, anda dapat dadu double!" seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project.** Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp= sharing

Additional:

ESSAY

1. Riset dan cite artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **
2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *
3. Sebutkan, dan jelaskan mengenai SOLID Principles *
4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

**** = 2 sumber**

*** = 1 sumber**

Gunakanlah sumber primary artikel/jurnal/buku, web hanya sebagai penunjang opini/argument

A. Source Code

MainActivity.kt XML

Tabel 1. Source Code MainActivity.kt XML

```
package com.example.xml

import android.os.Bundle
import android.widget.Button
import android.widget.ImageView
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val ivDice1 = findViewById<ImageView>(R.id.ivDice1)
        val ivDice2 = findViewById<ImageView>(R.id.ivDice2)
        val btnRoll = findViewById<Button>(R.id.btnRoll)
        val tvResult = findViewById<TextView>(R.id.tvResult)

        btnRoll.setOnClickListener {

            val dice1 = (1..6).random()
            val dice2 = (1..6).random()

ivDice1.setImageResource(getDrawableForDice(dice1))
ivDice2.setImageResource(getDrawableForDice(dice2))

            if (dice1 == dice2) {
                tvResult.text = "Selamat, anda dapat dadu double!"
            } else {
                tvResult.text = "Anda belum beruntung!"
            }
        }
    }
    private fun getDrawableForDice(value: Int): Int {
        return when (value) {
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
```

```
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }
    }
}
```

activity_main.xml

Tabel 2. Source Code activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#1a1a2e"
    android:gravity="center"
    android:orientation="vertical">

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <ImageView
            android:id="@+id/ivDice1"
            android:layout_width="150dp"
            android:layout_height="150dp"
            android:contentDescription="@null"
            android:src="@drawable/dice_0"
            android:padding="8dp"/>

        <ImageView
            android:id="@+id/ivDice2"
            android:layout_width="150dp"
            android:layout_height="150dp"
            android:contentDescription="@null"
            android:src="@drawable/dice_0"
            android:padding="8dp"/>

    </LinearLayout>

    <Space
        android:layout_width="wrap_content"
        android:layout_height="16dp"/>
```

```

    <Button
        android:id="@+id/btnRoll"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:backgroundTint="#c9b8e8"
        android:text="Roll"
        android:textColor="#1a1a2e"
        tools:ignore="HardcodedText"/>

    <Space
        android:layout_width="wrap_content"
        android:layout_height="16dp"/>

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text=""
        android:textColor="#ffffff"
        android:textSize="16sp"/>

</LinearLayout>
```

MainActivity.kt Compose

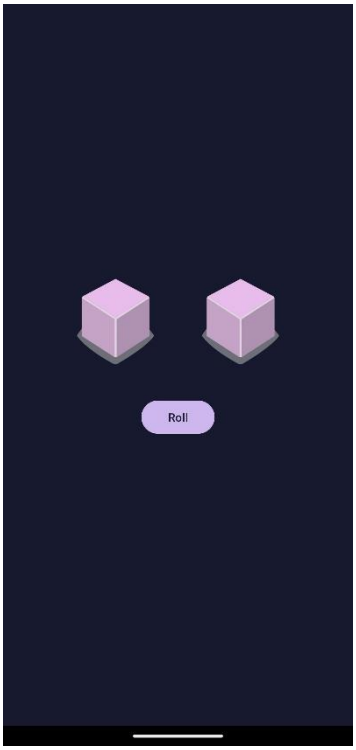
Tabel 3. Source Code MainActivity.kt Compose

```
package com.example.compose

import android.annotation.SuppressLint
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import
androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
```

```
import androidx.compose.ui.unit.sp
import android.os.Bundle
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            DiceRollerApp()
        }
    }
}

@SuppressLint("SetTextI18n")
@Composable
fun DiceRollerApp() {
    var dice1 by remember { mutableIntStateOf(0) }
    var dice2 by remember { mutableIntStateOf(0) }
    var resultMessage by remember { mutableStateOf("") }

    fun getDrawableForDice(value: Int): Int {
        return when (value) {
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(Color(0xFF1a1a2e)),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Row(
            horizontalArrangement = Arrangement.Center,
            modifier = Modifier.fillMaxWidth()
        ) {
            Image(
                painter = painterResource(
                    id = if (dice1 == 0) R.drawable.dice_0 else getDrawableForDice(dice1)
                ),
                contentDescription = "Dice 1",
```

```
                modifier = Modifier
                    .size(150.dp)
                    .padding(8.dp)
            )
            Image(
                painter = painterResource(
                    id = if (dice2 == 0) R.drawable.dice_0 else getDrawableForDice(dice2)
                ),
                contentDescription = "Dice 2",
                modifier = Modifier
                    .size(150.dp)
                    .padding(8.dp)
            )
        }

        Spacer(modifier = Modifier.height(16.dp))

        Button(
            onClick = {
                dice1 = (1..6).random()
                dice2 = (1..6).random()
                resultMessage = if (dice1 == dice2) {
                    "Selamat, anda dapat dadu double!"
                } else {
                    "Anda belum beruntung!"
                }
            },
            colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFc9b8e8)),
            shape = RoundedCornerShape(50)
        ) {
            Text(text = "Roll", color = Color(0xFF1a1a2e))
        }

        Spacer(modifier = Modifier.height(16.dp))

        Text(
            text = resultMessage,
            color = Color.White,
            fontSize = 16.sp
        )
    }
}
```

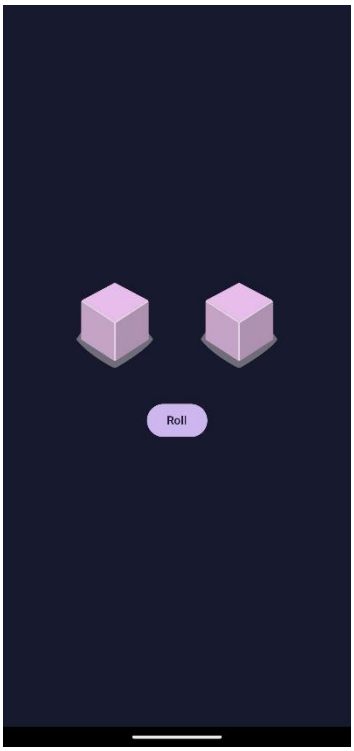
B. Output Program



Gambar 4. Screenshot Hasil Jawaban Soal 1 XML

Gambar 5. Screenshot Hasil Jawaban Soal 1 XML

Gambar 6. Screenshot Hasil Jawaban Soal 1 XML



Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose

Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose

Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

MainActivity.kt XML

File `MainActivity.kt` merupakan pusat kendali aplikasi pada versi XML. Class `MainActivity` mewarisi `AppCompatActivity`, dan proses pertama yang terjadi di dalam fungsi `onCreate()` adalah pemanggilan `setContentView(R.layout.activity_main)` yang menghubungkan file Kotlin ini dengan tampilan yang sudah didefinisikan di `activity_main.xml`. Setelah tampilan terhubung, empat komponen UI diambil menggunakan `findViewById` dan disimpan ke dalam variabel lokal. `ivDice1` dan `ivDice2` bertipe `ImageView` digunakan untuk menampilkan gambar dadu, `btnRoll` bertipe `Button` sebagai pemicu aksi, dan `tvResult` bertipe `TextView` sebagai tempat munculnya pesan hasil.

Seluruh logika permainan berjalan di dalam `btnRoll.setOnClickListener`. Saat tombol ditekan, dua angka acak antara 1 hingga 6 dibangkitkan menggunakan `(1..6).random()` dan disimpan ke variabel `dice1` dan `dice2`. Kedua angka tersebut langsung dikirim ke fungsi `getDrawableForDice()` yang mengembalikan resource gambar dadu yang sesuai, lalu diterapkan ke masing-masing `ImageView` melalui `setImageResource()`. Fungsi `getDrawableForDice()` dideklarasikan sebagai `private fun` di luar `onCreate()` menggunakan ekspresi `when` untuk memetakan setiap angka ke gambar drawable yang tepat. Setelah gambar diperbarui, percabangan `if-else` membandingkan nilai `dice1` dan `dice2` — jika sama maka `tvResult` menampilkan pesan double, jika berbeda menampilkan pesan belum beruntung langsung di layar melalui properti `.text`.

activity_main.xml

File `activity_main.xml` mendefinisikan seluruh struktur tampilan aplikasi. Berbeda dari `ConstraintLayout` yang umum digunakan, layout ini menggunakan `LinearLayout` sebagai root dengan orientasi vertikal sebagai pendekatan yang lebih sederhana dan mudah dipahami. Atribut `android:gravity="center"` memastikan semua elemen anak tersusun rapi di tengah layar, sementara warna latar belakang gelap `#1a1a2e` memberikan kesan visual

yang modern. Di dalam root layout, terdapat `LinearLayout` kedua berorientasi horizontal yang bertugas menyandingkan dua gambar dadu secara bersamaan. Kedua `ImageView` di dalamnya, yaitu `ivDice1` dan `ivDice2`, memiliki ukuran seragam `150dp x 150dp` dengan padding `8dp` di setiap sisinya agar tidak terlalu berhimpitan. Keduanya diinisialisasi dengan gambar `dice_0` sehingga saat aplikasi pertama dibuka, yang tampil adalah dua dadu kosong tanpa angka.

Untuk mengatur jarak antar elemen secara vertikal, digunakan komponen `Space` dengan tinggi `16dp` yang disisipkan di antara wadah dadu, tombol, dan teks hasil. Pendekatan ini lebih bersih dibandingkan menggunakan `margin` pada setiap komponen. Komponen `Button` dengan ID `btnRoll` ditampilkan dengan warna latar `#c9b8e8` melalui atribut `android:backgroundTint` dan teks berwarna gelap `#1a1a2e` agar kontras dan mudah dibaca. Di bagian paling bawah terdapat `TextView` dengan ID `tvResult` yang awalnya kosong, siap diisi oleh logika Kotlin ketika tombol Roll ditekan, dengan warna teks putih dan ukuran `16sp`.

MainActivity.kt Compose

File `MainActivity.kt` pada versi Jetpack Compose ini memiliki pendekatan yang berbeda dibandingkan versi XML. Class `MainActivity` mewarisi `ComponentActivity`, dan di dalam fungsi `onCreate()` hanya terdapat satu pemanggilan fungsi `setContent {}` yang langsung merender fungsi composable `DiceRollerApp()` ke layar. Tidak ada lagi proses `findViewById` seperti pada pendekatan XML karena UI dan logika dikelola dalam satu tempat yang sama.

Fungsi `DiceRollerApp()` menjadi inti dari seluruh aplikasi. Di sini dideklarasikan tiga variabel state, yaitu `dice1` dan `dice2` bertipe `mutableIntStateOf(0)` untuk menyimpan angka hasil roll, serta `resultMessage` bertipe `mutableStateOf("")` untuk menyimpan teks pesan. Ketiganya dibungkus dengan `remember` agar nilainya tidak hilang ketika UI digambar ulang. Inilah perbedaan mendasar paradigma deklaratif — developer cukup mengubah nilai state, dan tampilan akan menyesuaikan diri secara otomatis.

Fungsi lokal `getDrawableForDice()` dideklarasikan di dalam `DiceRollerApp()` dan bertugas mengonversi angka 1 hingga 6 menjadi referensi gambar drawable menggunakan ekspresi `when`. Susunan tampilan dibangun menggunakan `Column` yang memenuhi seluruh layar dengan `fillMaxSize()`, berlatar belakang warna gelap, dan seluruh isinya dipusatkan secara horizontal maupun vertikal. Di dalamnya terdapat `Row` yang menampung dua komponen `Image` secara berdampingan. Setiap `Image` mengevaluasi nilai state-nya — jika masih bernilai 0 maka menampilkan dadu kosong, dan jika sudah ada angka maka menampilkan gambar dadu yang sesuai.

Tombol `Button` berbentuk membulat penuh karena menggunakan `RoundedCornerShape(50)`, dengan warna latar lavender yang selaras dengan tema aplikasi. Ketika ditekan, kedua nilai dadu diperbarui secara acak dan `resultMessage` langsung diisi sesuai hasil perbandingan keduanya. Komponen `Text` di bagian bawah kemudian menampilkan pesan tersebut secara otomatis karena terikat langsung dengan nilai state `resultMessage`.

D. Essay

1. Riset dan cite artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan.

   Jawab:

   Dalam pengembangan antarmuka pengguna (UI), terdapat dua paradigma pemrograman utama yang sering dibandingkan, yaitu imperative programming dan declarative programming. Imperative programming adalah pendekatan di mana programmer mendefinisikan secara eksplisit setiap langkah yang harus dilakukan oleh sistem untuk mencapai hasil yang diinginkan. Endres et al. (2017) menjelaskan bahwa pada pendekatan imperatif, model proses mendefinisikan seluruh aktivitas yang harus dijalankan beserta urutan eksekusi dan aliran datanya. Dalam konteks pembuatan UI Android, pendekatan ini tercermin pada penggunaan XML layout, di mana developer secara manual mendefinisikan setiap elemen tampilan, mengatur listener, dan memperbarui UI secara eksplisit melalui kode.

Sebaliknya, declarative programming memungkinkan programmer untuk mendefinisikan *apa* yang ingin ditampilkan, bukan *bagaimana cara* menampilkannya. Hanus & Kluß (2009) memaparkan bahwa pendekatan deklaratif pada UI memisahkan aspek struktur, fungsionalitas, dan layout secara jelas, sehingga menghasilkan kode yang lebih ringkas, mudah dipahami, dan dapat digunakan kembali. Dalam pengembangan Android modern, pendekatan ini diwujudkan melalui Jetpack Compose, di mana developer cukup mendeskripsikan tampilan UI dan sistem secara otomatis menangani perubahan state yang terjadi.

Opini: Berdasarkan hasil riset di atas, saya lebih memilih paradigma declarative programming, khususnya Jetpack Compose, dalam pembuatan UI. Alasannya karena pendekatan deklaratif menghasilkan kode yang lebih ringkas dan mudah dipelihara. Dengan Jetpack Compose, perubahan pada state UI langsung tercermin pada tampilan tanpa perlu menulis banyak kode boilerplate seperti pada XML. Meskipun imperative programming dengan XML memberikan kontrol yang lebih eksplisit dan familiar bagi developer lama, kompleksitasnya cenderung meningkat seiring bertambahnya fitur aplikasi. Oleh karena itu, untuk pengembangan aplikasi mobile modern, pendekatan deklaratif dinilai lebih efisien dan produktif.

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java?
   Jawab:

   Kotlin dan Java sama-sama merupakan bahasa pemrograman berbasis Object-Oriented Programming (OOP) yang digunakan dalam pengembangan aplikasi Android. Meskipun keduanya berjalan di atas Java Virtual Machine (JVM), terdapat beberapa perbedaan mendasar dalam penerapan konsep OOP pada kedua bahasa ini. Pertama, dari sisi penanganan Null. Bose et al. (2018) menjelaskan bahwa salah satu kelemahan utama Java dalam OOP adalah rentan terhadap NullPointerException (NPE), yaitu ketika sebuah objek diakses namun bernilai null. Kotlin mengatasi hal ini dengan fitur Null Safety, di mana secara default semua variabel tidak boleh bernilai null kecuali dideklarasikan secara eksplisit, sehingga mengurangi risiko crash pada aplikasi.

   Kedua, dari sisi constructor. Kotlin mendukung primary constructor dan secondary constructor dalam satu class, sementara Java hanya mengenal constructor overloading biasa yang memerlukan lebih banyak baris kode untuk mencapai hasil yang sama. Ketiga, dari sisi extension function. Kotlin memungkinkan developer menambahkan fungsi baru ke sebuah class

tanpa harus mewarisi class tersebut, yang dikenal sebagai extension function. Fitur ini tidak tersedia di Java, sehingga Java mengharuskan penggunaan wrapper class atau inheritance untuk mencapai fungsionalitas serupa. Keempat, dari sisi conciseness. Kode Kotlin jauh lebih ringkas dibandingkan Java. Sebagai contoh, sebuah class yang memerlukan 50 baris kode di Java dapat ditulis hanya dalam beberapa baris di Kotlin karena Kotlin secara otomatis men-generate method seperti equals(), hashCode(), dan toString().

3. Sebutkan, dan jelaskan mengenai SOLID Principles.

   Jawab:

   SOLID merupakan akronim dari lima prinsip desain perangkat lunak yang diperkenalkan oleh Robert C. Martin (dikenal sebagai Uncle Bob). Yadati (2019) menjelaskan bahwa penerapan prinsip-prinsip ini dalam pengembangan aplikasi Android dapat menghasilkan kode yang lebih bersih, mudah dipelihara, dan skalabel.

   - Single Responsibility Principle (SRP): Sebuah class seharusnya hanya memiliki satu alasan untuk berubah, artinya setiap class hanya boleh memiliki satu tanggung jawab. Dalam konteks Android, prinsip ini mendorong pemisahan antara logika UI, logika bisnis, dan operasi data ke dalam class-class yang berbeda sehingga kode lebih terorganisir dan mudah dipahami.
   - Open/Closed Principle (OCP): Entitas perangkat lunak harus terbuka untuk ekstensi tetapi tertutup untuk modifikasi. Artinya, perilaku sebuah sistem dapat dikembangkan tanpa harus mengubah kode yang sudah ada. Dalam Kotlin, prinsip ini diterapkan melalui penggunaan polymorphism dan inheritance.
   - Liskov Substitution Principle (LSP): Objek dari sebuah superclass harus dapat digantikan dengan objek dari subclass-nya tanpa mempengaruhi kebenaran program. Prinsip ini memastikan bahwa subclass selalu mempertahankan perilaku yang diharapkan dari base class-nya, sehingga menghasilkan kode yang lebih andal dan dapat diprediksi.
   - Interface Segregation Principle (ISP): Client tidak boleh dipaksa untuk bergantung pada interface yang tidak mereka gunakan. Prinsip ini mendorong pembuatan interface yang lebih spesifik sesuai kebutuhan masing-masing client, sehingga menghasilkan kode yang lebih modular dan fleksibel.

- Dependency Inversion Principle (DIP): Modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah, keduanya harus bergantung pada abstraksi. Dalam pengembangan Android dengan Kotlin, prinsip ini diterapkan melalui dependency injection untuk mengelola ketergantungan antar komponen aplikasi.

4. Apa perbedaan antara Class dan Data Class pada Kotlin?

   Jawab:

   Dalam bahasa pemrograman Kotlin, terdapat dua jenis class yang sering digunakan yaitu class biasa dan data class. Keduanya memiliki perbedaan mendasar dalam hal tujuan penggunaan dan fitur yang disediakan secara otomatis oleh compiler. Class biasa digunakan untuk merepresentasikan objek yang memiliki perilaku (behavior) dan state yang kompleks. Pada class biasa, developer harus mendefinisikan sendiri method-method seperti equals(), hashCode(), toString(), dan copy() apabila diperlukan.

   Data class adalah class khusus yang dirancang khusus untuk menyimpan data. Jemerov & Isakova dalam buku Kotlin in Action (2023) menjelaskan bahwa ketika sebuah class dideklarasikan dengan keyword data, compiler Kotlin secara otomatis men-generate beberapa method penting, yaitu equals() untuk membandingkan nilai antar objek, hashCode() untuk penggunaan dalam collections, toString() untuk menampilkan isi objek secara readable, dan copy() untuk membuat salinan objek dengan kemungkinan mengubah sebagian nilainya.

   Perbedaan utama lainnya adalah pada cara perbandingan objek. Pada data class, perbandingan dilakukan berdasarkan nilai propertinya, sedangkan pada class biasa perbandingan dilakukan berdasarkan referensi memori objek tersebut. Dalam pengembangan Android, data class sangat cocok digunakan untuk merepresentasikan model data seperti response dari API, entitas database, atau objek yang hanya berfungsi sebagai wadah data.

E. Daftar Pustaka

[1] M. Hanus and C. Kluß, "Declarative Programming of User Interfaces," in *Proc. 11th International Symposium on Practical Aspects of Declarative Languages (PADL 2009)*, Springer LNCS 5418, 2009, pp. 16–30.

[2] C. Endres, U. Breitenbücher, M. Falkenthal, O. Kopp, F. Leymann, and J. Wettinger, "Declarative vs. Imperative: Two Modeling Patterns for the Automated Deployment of

Applications," in *Proc. 9th International Conference on Pervasive Patterns and Applications (PATTERNS 2017)*, Xpert Publishing Services, 2017, pp. 22–27.

[3] S. Bose, A. Kundu, M. Mukherjee, and M. Banerjee, "A Comparative Study: Java vs Kotlin Programming in Android Application Development," *International Journal of Advanced Research in Computer Science*, vol. 9, no. 3, pp. 41–45, May–Jun. 2018.

[4] N. S. P. K. Yadati, "Solid Principles in Android Development with Kotlin," *Journal of Scientific and Engineering Research*, vol. 6, no. 2, pp. 282–286, 2019.

[5] D. Jemerov and S. Isakova, *Kotlin in Action*, 2nd ed. Shelter Island, NY: Manning Publications, 2023.

TAUTAN GIT

Berikut adalah tautan untuk *source code* yang telah dibuat.

https://github.com/arinn-lab/Module-Mobile